Свёртка изображений и потоковая обработка

Сравнение последовательной, параллельной и агентной обработки

Михаил Тупчий 24.Б41

СПбГУ

09S.10.2026

Введение

- **Цель:** сравнить несколько подходов свертки изображений: последовательной, параллельной и агентной обработки.
- **Вопросы исследования:**
  - Даёт ли выигрыш параллелизм по строкам и столбцам?
  - Помогает ли агентная конвейерная модель?
  - Как влияет количество агентов-обработчиков?
  - Как моя реализация соотносится с библиотекой ImageSharp?

Экспериментальная установка

Аппаратная платформа

- CPU: Intel Core i5-12450H
- 8 физических ядер (12 потоков)

Программная реализация

- ImageSharp: загрузка и сохранение
- Свёртка: собственная

Тестовый набор

- 100 изображений JPEG, 4к формата

Методология замеров

- Сквозное время: весь вызов обработки, включая подготовку, чтение, свёртку и запись.
- 25 измерений на режим.
- Таймер: `Stopwatch`. Медиана и диапазон полного времени.

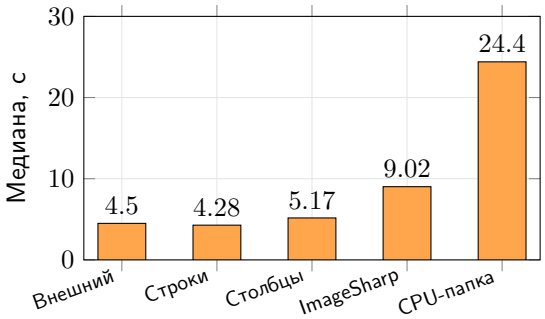
Исследуемые подходы

Подход	Уровень	Описание работы
1. Внешний параллелизм	По файлам	Внешний Parallel.ForEach. Внутри файла последовательно.
2. Вложенный: строки	Файлы и строки	Внешний цикл по файлам и Parallel.For по y внутри изображения.
3. Вложенный: столбцы	Файлы и столбцы	Внешний цикл по файлам и Parallel.For по x внутри изображения.
4. Агентная модель	Конвейер задач	Настраиваемые читатели, N CPU-вычислителей с последовательной свёрткой и писатели.
5. CPU-папка	По одному файлу	Чтение, последовательная свёртка и запись без параллелизма.
6. ImageSharp		Библиотечный вариант.

Результаты: сравнение базовых подходов

100 изображений 4K.

Подход	Медиана, с
CPU-папка	24.40
Внешний параллелизм	4.50
Вложенный: строки	4.28
Вложенный: столбцы	5.17
ImageSharp, BoxBlur	9.02



Последовательная CPU-папка: из чего складывается время

100 JPEG 4K.

Стадия для всей папки	Медиана суммы, с
Чтение JPEG и получение яркости пикселей	9.66
Последовательная CPU-свёртка	7.69
Подготовка результата и сохранение JPEG	8.09

Почему столбцы медленее

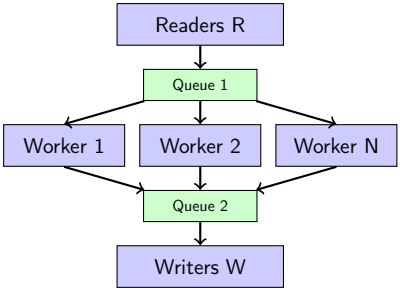
Опыт: та же свёртка, но во входе и результате рядом в памяти лежат пиксели одной строки либо одного столбца.

Рядом в памяти	Обход	Время, мс	LLC-промахи, млн
Пиксели одной строки	По строкам	18.74	0.23
Пиксели одной строки	По столбцам	36.37	8.77
Пиксели одного столбца	По столбцам	18.85	0.23
Пиксели одного столбца	По строкам	36.95	8.59

Вывод: при ядре 3×3 для 8 соседних результатов вдали от края нужны 30 входных пикселей: по строке — 3 строки по 10, по столбцу — 10 строк по 3. При хранении по строкам первый обход затрагивает меньше кэш линий и записывает результаты подряд. При хранении по столбцам преимущество получает столбцовый обход.

Агентная модель: архитектура конвейера

Структура конвейера



Роли агентов:

- **Reader**: пиксели в Queue 1
- **Workers**: свёртка → Queue 2
- **Writer**: сохранение из Queue 2

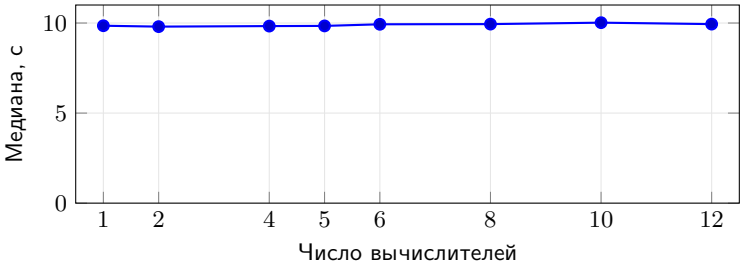
Программная реализация:

1. **Очереди данных:** используются BlockingCollection с лимитом в 100 элементов для предотвращения переполнения памяти.
2. **Читатели:** количество настраивается; каждый файл читает один агент.
3. **Обработчики (Workers):** N агентов выполняют последовательную свертку. Результаты поступают во вторую очередь.
4. **Писатели:** количество настраивается; каждый результат сохраняет один агент. По умолчанию $R = W = 1$.

Агентная модель: влияние числа воркеров

1 читатель, N вычислителей, 1 писатель; 25 измерений без профиля.

N	Медиана, с
1	9.85
2	9.80
4	9.83
5	9.84
6	9.93
8	9.94
10	10.02
12	9.94



CPU-конвейер: что показали ожидания

100 изображений 4K; медианы по 25 прогонам с профилем. R/W — читатели/писатели, N — вычислители.

N	R/W	Вся папка, с	Ждут вход, с	Ждут выход, с	Результат изменения
5	1/1	9.95	41.26	0.01	Базовый вариант
5	1/2	9.85	40.82	0.01	Почти без изменений
5	2/1	7.62	3.53	22.94	Почти без изменений
5	2/2	6.00	18.90	0.01	Ускорение
5	3/3	4.73	9.57	0.01	Ускорение
4	4/4	4.24	0.43	0.01	Лучшее в этой серии
2	5/5	5.28	0.20	0.01	Мало вычислителей

Ждут вход: очередь ещё не получила подготовленное изображение. **Ждут выход:** очередь результатов заполнена, писатель не успевает освобождать её.

ImageSharp: где тратится время внутри BoxBlur

Что измеряли: BoxBlur 3×3 одного серого изображения 3840×2160, заранее загруженного в память.

Как ImageSharp выполняет BoxBlur серого изображения:

Вход: `Image<L8>` — всё изображение; `L8` хранит яркость одного пикселя.
По горизонтали: `L8` → `Vector4` (RGBA) → усреднение → `L8`.
По вертикали: `L8` → `Vector4` (RGBA) → усреднение → `L8`.

Где находился процессор внутри BoxBlur	Доля выборок CPU
Преобразование результата обратно в серый пиксель	46.9%
Сложение и умножение четырёх компонент	18.1%
Циклы горизонтального и вертикального проходов	22.8%
Обработка прозрачности	5.2%

Visual Studio CPU Usage: 62 400 выборок за 100 вызовов. Проценты показывают распределение выборок внутри фильтра.

Сводная таблица всех подходов

100 изображений 4K.

Метод	Медиана, с	Мин.–макс., с
CPU-папка	24.40	23.90–26.26
Внешний цикл	4.50	4.44–4.66
Вложенный: строки	4.28	4.19–4.46
Вложенный: столбцы	5.17	4.97–5.39
ImageSharp	9.02	8.91–9.19
Лучшие CPU-агенты: 4 вычислителя, R4/W4	**4.21**	**4.04–4.37**

R/W — читатели/писатели.

Выводы: ответы на вопросы исследования

1. Даёт ли выигрыш параллелизм по строкам и столбцам?

- Да, по строкам дает, но не значиительный, по столбцам же дольше. Строки: 4.28 с; столбцы: 5.17 с.

2. Помогает ли агентная модель?

- Да. Если конфигурировать всех агентов, а не только агентов для свертки.

3. Как влияет количество агентов-обработчиков?

- В этих опытах требовалось ускорить и подготовку входа, и сохранение. Одно увеличение числа вычислителей проблему не решало.

4.Как моя реализация соотносится с библиотекой ImageSharp?

- ImageSharp уступает всем реализациям, кроме CPU папки.

Спасибо за внимание!